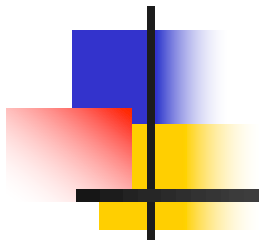
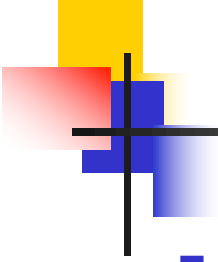


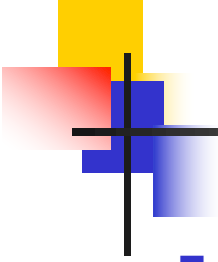
7.5 Otros aspectos





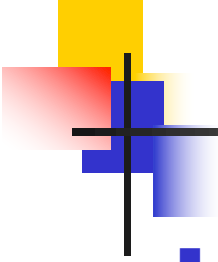
Índice

- Recargas
- Política Same-origin y CORS



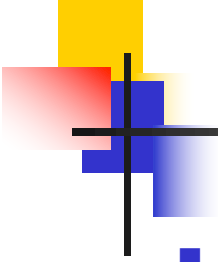
Recargas

- Las recargas del frontend las experimenta habitualmente el desarrollador cuando modifica el código del frontend servido por **npm run dev**
- En las aplicaciones React creadas con Vite
 - [1] Cuando se modifica el código de un componente React (archivo **.jsx**)
 - Se recarga el código de ese componente automáticamente en el navegador
 - **Además, en muchos casos, si el componente tenía estado local (useState), el estado se conserva => muy cómodo**
 - [2] Pero, cuando se modifica un archivo **.js**
 - Todo el código del frontend se recarga automáticamente en el navegador
 - **Todas las variables pierden su valor**
 - Estado de Redux, estado local de los componentes y resto de variables del frontend



Recargas

- Si no se hace nada especial, cuando se produce [2]
 - Los datos de perfil de usuario y la información del carrito (que se habían devuelto al autenticarse) ya no estarán en el estado de Redux
 - El desarrollador (que está probando la aplicación), tendría que volver a autenticarse
- Cuando se produce una recarga completa del frontend (caso [2]), en particular, se vuelve a renderizar el componente **App**
 - Tanto en pa-shop, como en la plantilla de la práctica, si el usuario estaba autenticado, **App** causa que se envíe un POST a `/users/loginFromServiceToken`, para reautenticar al usuario automáticamente a partir del token JWT que había obtenido cuando se había autenticado explícitamente
 - **POST /users/loginFromServiceToken** devuelve lo mismo que **POST /users/login**
 - Otra vez el mismo token JWT
 - Los datos del perfil del usuario y la información del carrito
 - Se evita que el desarrollador tenga que volver a autenticarse
 - El estado local de los componentes se pierde

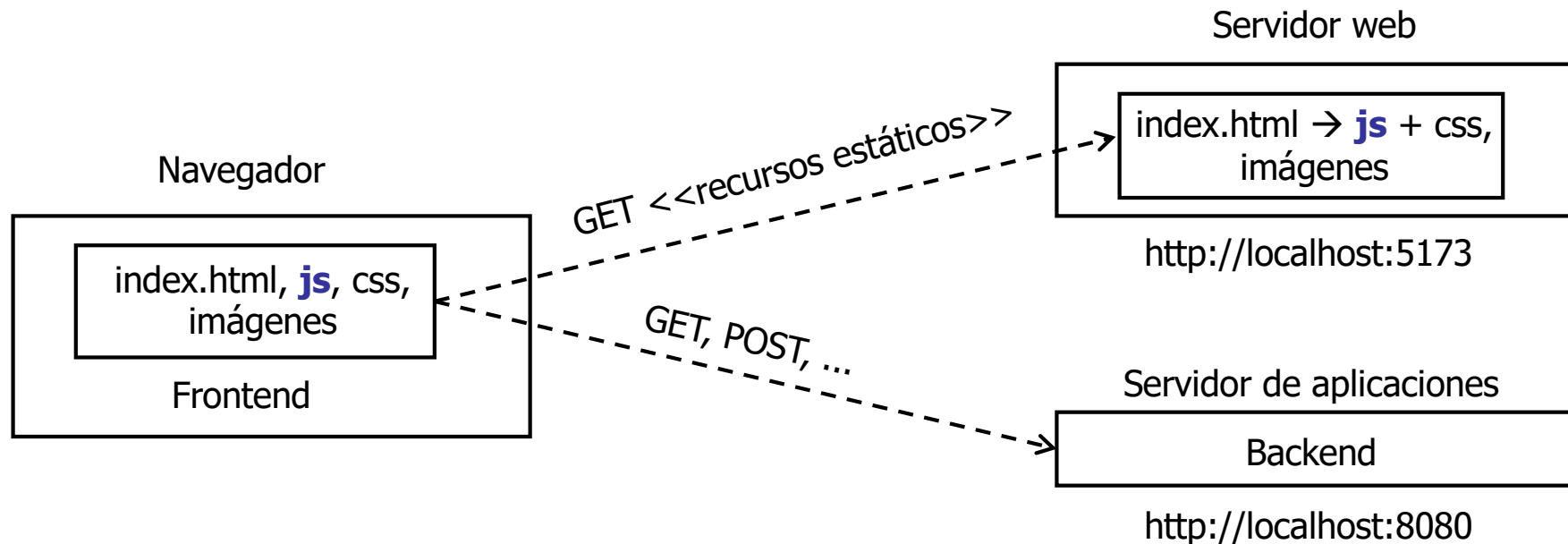


Recargas

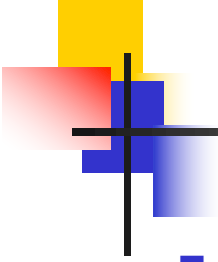
- Pero ..., ¿dónde se guarda el token JWT cuando el usuario se autentica (explícitamente o implícitamente)?
- Si se guardase en un objeto JavaScript, se habría perdido al recargar el frontend
- Para hacer frente a este problema, el token JWT se guarda usando el almacenamiento Web proporcionado por el navegador
 - <https://developer.mozilla.org/en-US/docs/Web/API/Storage>
 - Los navegadores modernos ofrecen la API `Storage` que permite guardar pares <clave (String), valor (String)>
- El navegador proporciona dos objetos que implementan **Storage**
 - **localStorage**
 - Los datos almacenados no tienen fecha de expiración
 - **sessionStorage**
 - Los datos están vinculados a la página/tab
 - Si la página/tab se cierra, los datos se eliminan
- En el caso de estudio, el token JWT se guarda en **sessionStorage**

Política Same-origin y CORS

- En la arquitectura empleada en el caso de estudio, el código JavaScript del frontend se descarga de una ubicación distinta a la del backend



- Por seguridad, los navegadores implementan la política **Same-origin**
 - Un script descargado de un "origen" no puede interactuar con un recurso de otro "origen"
 - Se considera que dos URLs tienen el mismo origen si emplean el mismo protocolo, host y puerto
 - https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy
 - En consecuencia, si no se hace nada especial, la anterior arquitectura no es viable



Política Same-origin y CORS

- La emergencia de las aplicaciones Web SPA hizo que surgiera el mecanismo estándar CORS (Cross-Origin Resource Sharing)
 - En las respuestas del backend se añaden cabeceras que especifican los permisos que se le otorgan a un script que corra en el navegador para interactuar con el backend
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
 - Por sencillez, el backend del caso de estudio permite que cualquier script procedente de cualquier origen pueda interactuar con él
 - Código fuente del backend: `SecurityConfig.corsConfigurationSource`